CS 6210: Matrix Computations

Hessenberg QR iteration

David Bindel

2025-10-29

Road map

Last lecture had two major themes:

1. We discussed power iteration and various ways of speeding it up through spectral transformations, particularly shift-invert transformations. We also pointed out that Rayleigh quotients make excellent shifts (albeit with extra factorization costs).
2. We discussed orthogonal iteration (aka simultaneous iteration).

This is a promising set of ingredients, but:

1. We are often as interested (or more) in eigenvalues as we are in eigenvectors, but our iterations so far are focused on the vectors and subspaces.
2. One step of Rayleigh quotient iteration seems to cost $O(n^3)$. So does one step of orthogonal iteration. This is very expensive.
3. We know how to use shifts to accelerate the convergence of power iteration (with Rayleigh quotients as a good source of shifts). But while this gives good local convergence, it's unclear how to get good global convergence. It's also unclear what we should do to converge to complex conjugate eigenpairs of real matrices.

In this lecture, we will treat each of these issues in turn.

Orthogonal iteration re-interpreted, take 1

Consider the orthogonal iteration as repeatedly applying a mapping $Q \mapsto Q'$ given by

$$AQ = Q'R.$$

Last time, we observed that

$$AQ_{:,1:k} = Q'_{:,1:k}R_{1:k,1:k},$$

i.e. this is equivalent to doing orthogonal iteration with subspaces of size k for $k = 1, 2, ...$

When Q is a unitary matrix, we can take the inverse conjugate transpose of both sides to get that

$$A^{-*}Q = Q'R^{-*}.$$

Now using the lower triangularity of R^{-*}, we observe that

$$A^{-*}Q_{:,k:n} = Q'_{:,k:n}R^{-*}_{k:n,k:n}$$

for $k = 1, 2, ...$ That is, just as the first k columns are undergoing subspace iteration with A, the last k columns are undergoing subspace iteration with A^{-*}. In particular, the last column of Q is undergoing power iteration with a (conjugate) inverse transform! And if we want to apply a (conjugtate) shift-invert transform, we need only consider iteration with

$$Q \mapsto Q' \text{ via } (A - \sigma I)Q = Q'R.$$

Orthogonal iteration re-interpreted, take 2

Recall the Rayleigh quotient

$$\rho_A(u) = \frac{u^*Au}{u^*u}.$$

When u is an eigenvector, the Rayleigh quotient is the associated eigenvalue. The block analogue of the Rayleigh quotient for U with orthonormal columns is

$$P_A(U) = U^*AU,$$

and f the columns of U span an invariant subspace, we have $AU = UP_A(U)$. If U is the unitary factor in the Schur form $AU = UT$, then we have $P_A(U) = T$ is the upper triangular factor.

Now, consider one step of orthogonal iteration (aka subspace iteration aka simultaneous iteration) as a mapping $\underline{Q} \mapsto \underline{Q'}$ given by $A\underline{Q} = \underline{Q'}R$, and let Q be such that $\underline{Q'} = \underline{Q}Q$. As the iteration proceeds, the columns of Q converge to nested invariant subspaces for eigenvalues of different magnitude. What happens to the associated block Rayleigh quotients? Observe that

$$P_A(\underline{Q}) = \underline{Q}^*A\underline{Q} = (\underline{Q}^*\underline{Q'})R = QR$$
$$P_A(\underline{Q'}) = \underline{Q'}^*(A\underline{Q}\underline{Q}^*)\underline{Q'} = RQ$$

Therefore, we can get from $P_A(\underline{Q})$ to $P_A(\underline{Q}')$ via

$$P_A(\underline{Q}) = QR, \quad P_A(\underline{Q}') = RQ.$$

This mapping, where we compute a QR factorization and then multiply the factors in the reverse order, is the basic *QR iteration*.

Hessenberg matrices and QR steps in $O(n^2)$

A matrix H is said to be *upper Hessenberg* if it has nonzeros only in the upper triangle and the first subdiagonal. For example, the nonzero structure of a 5-by-5 Hessenberg matrix is

$$\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \end{bmatrix}.$$

For any square matrix A, we can find a unitarily similar Hessenberg matrix $H = Q^*AQ$ by applying Householder transformations on the left and right in a symmetric fashion.

A Hessenberg matrix H is very nearly upper triangular, and is an interesting object in its own right for many applications. For example, in control theory, one sometimes would like to evaluate a *transfer function*

$$h(s) = c^T(sI - A)^{-1}b + d$$

for many different values of s. Done naively, it looks like each each evaluation would require $O(n^3)$ time in order to get a factorization of $sI - A$; but if $H = Q^*AQ$ is upper Hessenberg, we can write

$$h(s) = (Qc)^*(sI - H)^{-1}(Qb) + d,$$

and the Hessenberg structure of $sI - H$ allows us to do Gaussian elimination on it in $O(n^2)$ time. Note that this also means that we can do shift-invert power method steps on H in $O(n^2)$ time!

Just as it makes it cheap to do Gaussian elimination, the special structure of the Hessenberg matrix also makes the Householder QR routine very economical. The Householder reflection computed in order to introduce a zero in the $(j+1, j)$ entry needs only to operate on rows j and $j+1$. Therefore, we have

$$Q^*H = W_{n-1}W_{n-2}\ldots W_1H = R,$$

where W_j is a Householder reflection that operates only on rows j and $j+1$. Computing R costs $O(n^2)$ time, since each W_j only affects two rows ($O(n)$ data). Now, note that

$$RQ = R(W_1W_2\ldots W_{n-1});$$

that is, RQ is computed by an operation that first mixes the first two columns, then the second two columns, and so on. The only subdiagonal entries that can be introduced in this process lie on the first subdiagonal, and so RQ is again a Hessenberg matrix. Therefore, one step of QR iteration on a Hessenberg matrix results in another Hessenberg matrix, and a Hessenberg QR step can be performed in $O(n^2)$ time.

Shifting gears

Henceforth, we consider Hessenberg iterates $A^{(k)}$ that we are trying to drive toward the quasi-triangular factor in a real Schur form.

The connection from inverse iteration to orthogonal iteration (and thus to QR iteration) gives us a way to incorporate the shift-invert strategy into QR iteration: simply run QR on the matrix $A - \sigma I$, and the (n, n) entry of the iterates (which corresponds to a Rayleigh quotient with an increasingly-good approximate row eigenvector) should start to converge to $\lambda - \sigma$, where λ is the eigenvalue nearest σ. Put differently, we can run the iteration

$$\begin{aligned} Q^{(k)} R^{(k)} &= A^{(k-1)} - \sigma I \\ A^{(k)} &= R^{(k)} Q^{(k)} + \sigma I. \end{aligned}$$

If we choose a good shift, then the lower right corner entry of $A^{(k)}$ should converge to the eigenvalue closest to σ in fairly short order, and the rest of the elements in the last row should converge to zero.

The shift-invert power iteration converges fastest when we choose a shift that is close to the eigenvalue that we want. We can do even better if we choose a shift *adaptively*, which was the basis for running Rayleigh quotient iteration. The same idea is the basis for the *shifted QR iteration*:

$$\begin{aligned} Q^{(k)} R^{(k)} &= A^{(k-1)} - \sigma_k I \\ A^{(k)} &= R^{(k)} Q^{(k)} + \sigma_k I. \end{aligned}$$

This iteration is equivalent to computing

$$\begin{aligned} \underline{Q}^{(k)} \underline{R}^{(k)} &= \prod_{j=1}^{n} (A - \sigma_j I) \\ A^{(k)} &= (\underline{Q}^{(k)})^* A (\underline{Q}^{(k)}) \\ \underline{Q}^{(k)} &= Q^{(k)} Q^{(k-1)} \ldots Q^{(1)}. \end{aligned}$$

What should we use for the shift parameters σ_k? A natural choice is to use $\sigma_k = e_n^* A^{(k-1)} e_n$, which is the same as $\sigma_k = (\underline{Q}^{(k)} e_n)^* A (\underline{Q}^{(k)} e_n)$, the Rayleigh quotient based on the last column of $\underline{Q}^{(k)}$. This simple shifted QR iteration is equivalent to running Rayleigh iteration starting from an initial vector of e_n, which we noted before is locally quadratically convergent.

Double trouble

The simple shift strategy we described in the previous section gives *local* quadratic convergence, but it is not *globally* convergent. As a particularly pesky example, consider what happens if we want to compute a complex conjugate pair of eigenvalues of a real matrix. With our simple shifting strategy, the QR iteration never produce a complex iterate, a complex shift, or a complex eigenvalue. The best we can hope for is that our initial shift is closer to both eigenvalues in the conjugate pair than it is to anything else in the spectrum; in this case, we will most likely find that the last two columns of $\underline{Q}^{(k)}$ are converging to a basis for an *invariant row subspace* of A, and the corresponding eigenvalues are the eigenvalues of the trailing 2-by-2 sub-block.

Fortunately, we know how to compute the eigenvalues of a 2-by-2 matrix! This suggests the following shift strategy: let σ_k be one of the eigenvalues of $A^{(k)}(n-1:n, n-1:n)$. Because this 2-by-2 problem can have complex roots even when the matrix is real, this shift strategy allows the possibility that we could converge to complex eigenvalues. On the other hand, if our original matrix is real, perhaps we would like to consider the *real* Schur form, in which U is a real matrix and T is block diagonal with 1-by-1 and 2-by-2 diagonal blocks that correspond, respectively, to real and complex eigenvalues. If we shift with *both* roots of $A^{(k)}(n-1:n, n-1:n)$, equivalent to computing

$$\begin{aligned} Q^{(k)}R^{(k)} &= (A^{(k-1)} - \sigma_{k+}I)(A^{(k-1)} - \sigma_{k-}) \\ A^{(k)} &= (Q^{(k)})^* A^{(k-1)} Q^{(k)}. \end{aligned}$$

There is one catch here: even if we started with $A^{(0)}$ in Hessenberg form, it is unclear how to do this double-shift step in $O(n^2)$ time!

The following fact will prove our salvation: if we Q and V are both orthogonal matrices and Q^TAQ and V^TAV are both (unreduced) Hessenberg[1]) and the first column of Q is the same as the first column of V, then all successive columns of Q are unit scalar multiples of the corresponding columns of V. This is the *implicit Q theorem*. Practically, it means that we can do any sort of shifted QR step we would like in the following way:

1. Apply as a similarity any transformations in the QR decomposition that affect the leading submatrix (1-by-1 or 2-by-2).
2. Restore the resulting matrix to Hessenberg form without further transformations to the leading submatrix.

In the first step, we effectively compute the first column of Q; in the second step, we effectively compute the remaining columns. Certainly we compute *some* transformation with the right leading column; and the implicit Q theorem tells us that any such transformation is basically the one we would have computed with an ordinary QR step.

[1] An unreduced Hessenberg matrix has no zeros on the first subdiagonal.

Last time, we discussed the Wilkinson strategy of choosing as a shift one of the roots of the trailing 2-by-2 submatrix of $A^{(k)}$ (the one closest to the final entry). We also noted that if we want to convert to *real* Schur form, the Wilkinson shift has the distinct disadvantage that it might launch us into the complex plane. The Francis shift strategy is to simultaneously apply a complex conjugate pair of shifts, essentially computing two steps together:

$$\begin{aligned} Q^{(k)}R^{(k)} &= (A^{(k-1)} - \sigma_k I)(A^{(k-1)} - \bar{\sigma}_k I) \\ &= (A^{(k-1)})^2 - 2\Re(\sigma_k)A^{(k-1)} + |\sigma_k|^2 I \\ A^{(k)} &= (Q^{(k)})^* A^{(k-1)} (Q^{(k)}). \end{aligned}$$

When the Wilkinson shift is real, we let σ_k be the same as the Wilkinson shift; when the Wilkinson strategy leads to a conjugate pair of possible shifts, we use both, maintaining efficiency by doing the steps *implicitly*. Let's now make this implicit magic a little more explicit by building code for an implicit double-shift QR step.

Our first step will be to construct the polynomial associated with the Francis double-shift. In the case where the trailing 2-by-2 submatrix (or 2-by-2 block Rayleigh quotient, if one prefers) has a complex pair of eigenvalues, we just use its characteristic polynomial. Otherwise, we use the polynomial associated with two steps with a Wilkinson shift.

The Francis double-shift strategy gives us coefficients b_k and c_k for a quadratic function $s_k(z) = z^2 + b_k z + c_k$. We now want to compute

$$\begin{aligned} Q^{(k)}R^{(k)} &= s_k(A^{(k-1)}) = (A^{(k-1)})^2 + b_k A^{(k-1)} + c_k I \\ A^{(k)} &= (Q^{(k)})^* A^{(k-1)} (Q^{(k)}). \end{aligned}$$

The trick is to realize that all the iterates $A^{(k)}$ are Hessenberg, and the Hessenberg form for a matrix is usually unique (up to signs). Therefore, we compute the first Householder transformation W in a QR factorization of $s_k(A^{(k)}$ explicitly. The first column of $Q^{(k)}$ is the same as the first column of W. The remaining columns of $Q^{(k)}$ can be determined by the requirement that $A^{(k)}$ is in Hessenberg form. We compute them implicitly by applying the usual Hessenberg reduction algorithm to $B = WA^{(k-1)}W$, taking advantage of the fact that B has special structure to do $O(n^2)$ work. Each step of the reduction moves a "bulge" down the diagonal by one.

In the LAPACK codes, the Francis double-shift strategy is mixed with some "exceptional shifts" that occur every few iterations. These exceptional shifts serve to keep the algorithm from getting stuck in certain pathological situations (e.g. a cyclic permutation matrix).

Deflation

A sequence of implicit doubly-shifted QR steps with the Francis shift will usually give us rapid convergence of a trailing 1-by-1 or 2-by-2 submatrix to a block of a Schur factorization. As

this happens, the trailing row (or two rows) becomes very close to zero. When the values in these rows are close enough to zero, we *deflate* by setting them equal to zero. This corresponds to a small perturbation to the original problem.

More careful deflation criteria are usually used in practice; see the book. This criterion at least corresponds to small normwise perturbations to the original problem, but it may result in less accurate estimates of small eigenvalues than we could obtain with a more aggressive criterion.

Stability of the method

Each step of the implicitly double-shifted QR iteration changes the matrix only with orthogonal transformations (which are perfectly conditioned) or deflations. Hence, the QR iteration is backward stable. However, this is *not* the same as saying that the method is forward stable! For forward stability, the conditioning of the eigenvalues is critical, and multiple (or nearly multiple) eigenvalues of multiplicity m usually inherit an $O(\epsilon^{1/m})$ error, as we saw in our earlier discussion of sensitivity.

The intermediate computations in the QR code as given above are prone to scaling problems, and so the basic QR codes in LAPACK (`dlahqr`) uses a more careful construction of a scaled copy of the first Householder transformation.

The state of the art

The current state of the art in QR iterations is the LAPACK code `dgehqr` written by Ralph Byers, which is based on an award-winning set of papers by Braman, Byers, and Mathias. This code uses the following general strategy:

1. Run the basic QR iteration to find the eigenvalues of a trailing $b \times b$ submatrix. Apply the transformations to the whole matrix, resulting in a "spike" to the left of the triangularized portion.
2. Look for converged eigenvalues in the trailing submatrix by analyzing the "spike" to find small elements. Deflate any eigenvalues found (and there may be several). This is called *aggressive early deflation*.
3. Use several of the remaining eigenvalues from the Rayleigh quotient block as a sequence of successive shifts. These can be run simultaneously by chasing a sequence of closely-spaced bulges down the main diagonal. The similarity transformations associated are applied in a blocky way to get good cache performance.